NED University of Engineering and Technology
Computer & Information Systems Engineering

Muhammad Ahmed Qazi
CS-24045

Lab Session 01

CS-326: CCN – Lab Workbook Exercises

Question 01:

```
\documentclass[12pt,a4paper] {article}
\usepackage[utf8] {inputenc}
\title{Chapai - A Print Management
System}
\author{Muhammad Ahmed Qazi}
\date{September 2026}
\begin{document}
\maketitle
\begin{abstract}
This article specifies the software
requirements for a smart printing
management system to be deployed in a
university campus environment.
\end{abstract}
\section{Introduction}
Traditional paper document printing
architectures on university campuses
represent a significant logistical
bottleneck, operational inefficiency,
and economic vulnerability.
To overcome these dificiencies,
"Chapai" introduces an automated,
transactionally secure,
hardware-aware, and priority-driven
smart campus printing management
network.
\subsection{User Requirements}
Following are the requirements that
our system fulfils:
\begin{enumerate}
    \item \textbf{Authentication:} A
user should be able to create an
account and login to it.
    \item \textbf{Print:} A user
(student or faculty) must be able to
send a document to be printed with the
desired configuration to our printing
kiosks.
    \item \textbf{Manage credit:} A
user should be able to do the
following:
    \begin{itemize}
        \item See the balance they
have in their account.
        \item Add some balance to
their account.
        \item See their transaction
history.
    \end{itemize}
    \item \textbf{Tracking:} A user
should be able to track the status of
their document whether it has been
queued, being printed, or ready for
collection.
\end{enumerate}
\section{Backend and Front-End Tools}
Our system uses the following tools to
lay the foundation of our system, i.e.
\textit{back-end tools}:
\begin{itemize}
    \item \textbf{Database Management
System:} \textit{Oracle Database 21c
Express Edition (XE)} with
\textit{Oracle SQL Developer as GUI}.
    \item \textbf{Backend
Architecture:} \textit{NestJS
Framework}, an enterprise-grade
TypeScript framework built atop
Node.js, implementing a strict modular
structure and managing standard Oracle
client database drivers via optimised
persistence layers.
    \item \textbf{Containerization
Subsystem:} Podman Container Engine,
utilised to sandbox the Oracle databse
environment, ensuring complete
reproducibility across developer
workstations and preventing operating
system dependency pollution.
\end{itemize}
The following tools are used to build
the interfaces for our system, i.e.
\textit{front-end tools}:
\begin{itemize}
    \item \textbf{Web Interface:}
\textit{Angular Framework}.
    \item \textbf{Mobile Interface:}
\textit{Ionic Framework} combined with
\textit{Angular 17}.
\end{itemize}
\section{Software Attributes}
Our print management system has the
following software attributes:
\begin{itemize}
    \item \underline{Functionality}:
print job submission, authentication,
account balance and transactions.
    \item \underline{Usability}: the
system has two distinct interface for
```

```
a user and an operator which are both
designed to accommodate the technical
ability of the user.
    \item \underline{Portability}:
this system is both deployed as a
desktop website and an Android mobile
application.
\end{itemize}
\section{Conclusion}
The Chapai platform successfully
demonstrates a highly functional,
enterprise-grade, three-tier
application designed to modernize and
streamline printing operations across
the NED University campus. By
replacing inefficient, manual printing
queues and unverified cash
transactions with a centralized
digital ledger and intelligent
hardware routing, the system addresses
core logistical bottlenecks for both
students and administration.
```

Question 02:

```
\documentclass[12pt,a4paper] {report}
\usepackage[utf8] {inputenc}
\usepackage{graphicx}
\graphicspath{{img/}}
\title{Basic Functionality of an
IoT-Based Health Monitoring System}
\author{Muhammad Ahmed Qazi}
\date{September 2026}
\begin{document}
\maketitle
\tableofcontents
\chapter{Introduction}
An \textbf{Internet of Things (IoT)
based Health Monitoring System} is an
interconnected network of smart
devices designed to
\textbf{continuously track, transmit,
and analyze} patient health metrics in
real time. This technology serves as a
vital bridge in modern healthcare,
delivering key benefits across several
critical areas:
\begin{itemize}
    \item \textbf{Remote Patient
Monitoring:} Enables continuous
clinical oversight from the comfort of
a patient's home.
    \item \textbf{Chronic Disease
Management:} Provides long-term
tracking for health conditions
requiring persistent vigilance.
The platform heavily leverages
advanced capabilities inherent to the
Oracle 21c ecosystem. Complex business
operations, such as simultaneous
balance deductions and ledger updates,
are securely encapsulated inside
PL/SQL stored procedures, guaranteeing
strict transactional atomicity.
Database triggers autonomously
maintain referential integrity across
unlinked relational bridges, while
highly optimized SQL views dynamically
flatten complex user hierarchies to
provide real-time analytical
reporting. By enforcing data isolation
at the database layer and controlling
access through a robust NestJS API,
the architecture ensures a highly
secure and highly available production
environment.
\end {document}

    \item \textbf{Elderly Care:}
Enhances independence and safety for
senior citizens without requiring
frequent, stressful hospital visits.
\end{itemize}
\chapter{System Architecture and
Functionality}
\section{System Workflow}
The basic functionality of the system
relies on a seamless flow of data from
the patient to the healthcare provider
through four main operational stages.
\begin{figure}[h]
    \centering

\includegraphics[width=0.75\textwidth]
{Architecture-of-Healthcare-in-IoT.jpg
}
    \caption{This is the image
description.}
    \label{fig:iot_arch}
\end{figure}
\subsection{Data Acquisition}
Biological sensors—either wearable or
implantable—continuously measure vital
physiological parameters from the
patient:
\begin{itemize}
    \item \textbf{Body Temperature
Sensors:} Track sudden
thermoregulatory fluctuations or fever
spikes.
    \item \textbf{Pulse Oximeters:}
Measure heart rate and blood oxygen
saturation levels.
```

```
    \item \textbf{Electrocardiogram (ECG) Sensors:} Monitor cardiac rhythm and detect potential arrhythmias.
\end{itemize}

\subsection{Data Processing and Transmission}
Raw physiological signals gathered by the sensors are directed to a central processing unit:
\begin{itemize}
    \item \textbf{Microprocessing Unit:} Hardware such as the \textit{ESP32, Arduino, or Raspberry Pi} filters, calibrates, and digitizes raw biological signals.
    \item \textbf{Communication Protocols:} Processed data is transmitted securely via \textit{Wi-Fi, Bluetooth Low Energy (BLE), or Zigbee} to a local gateway or directly to the internet.
\end{itemize}
\subsection{Cloud Storage and Analytics}
Once transmitted, the data enters a secure cloud infrastructure:
\begin{itemize}
    \item \textbf{Historical Storage:} Patient health metrics are archived securely to build accessible, long-term medical records.
    \item \textbf{Automated Analytics:} Machine learning algorithms process arriving data streams in real time to detect subtle anomalies, such as irregular heart rhythms or acute stress spikes.
\end{itemize}

\chapter{Advantages and Applications}
\section{Key Advantages}
Integrating IoT architecture into medical monitoring provides several transformative advantages:
\begin{itemize}
    \item \textbf{Reduced Hospital Readmissions:} Proactive monitoring prevents medical complications before they escalate into acute emergency stays.
    \item \textbf{Cost Efficiency:} Decreases overall healthcare expenditures by minimizing unnecessary in-person consultations and travel.
    \item \textbf{Proactive Care Paradigm:} Shifts medical practice from reactive treatment to continuous, preventive intervention.
\end{itemize}
\section{Primary Applications}
The system demonstrates significant utility across diverse medical domain scenarios:
\begin{itemize}
    \item \textbf{Elderly Remote Care:} Supports senior citizens living independently by tracking vitals and providing continuous emergency signaling.
    \item \textbf{Cardiovascular Monitoring:} Tracks heart rate variability and blood pressure for high-risk cardiac patients.
    \item \textbf{Post-Operative Recovery:} Ensures patients recovering from surgical procedures remain stable following early hospital discharge.
\end{itemize}
\chapter{Conclusion}
In summary, an IoT health monitoring system automates vital sign monitoring to establish continuous vigilance over patient health. By uniting sensor technology, wireless networking, and cloud computing, it provides an efficient, cost-effective, and deeply proactive foundation for modern digital healthcare.
\end{document}
```

Question 03:

```
\documentclass[12pt,a4paper] {article}
\usepackage[utf8] {inputenc}
\usepackage{amsmath}
\title{Requirement Specification for a Linear Equation Calculator}
\author{Muhammad Ahmed Qazi}
\date{September 2026}
\begin{document}
\maketitle

\begin{abstract}
This article presents an initial draft of the software requirements for a calculator application capable of performing arithmetic operations and solving basic linear equations.
\end{abstract}
\section{Introduction}
Conventional pocket calculators evaluate arithmetic expressions but
```

Muhammad Ahmed Qazi
CS-24045

```
cannot resolve an unknown quantity
presented in equation form. The
proposed calculator extends standard
arithmetic functionality with a
symbolic solver for basic linear
equations, allowing a user to enter an
equation directly and obtain its
solution.
\subsection{User Requirements}
Following are the requirements that
our system fulfils:
\begin{enumerate}
    \item \textbf{Arithmetic:} A user
must be able to evaluate expressions
involving addition, subtraction,
multiplication and division.
    \item \textbf{Equation entry:} A
user must be able to enter an equation
containing an equality sign and one or
more unknowns.
    \item \textbf{Solving:} The system
must determine the value of each
unknown and report the following:
    \begin{itemize}
        \item A unique solution, where
one exists.
        \item That the equation holds
for every value of the unknown.
        \item That the equation has no
solution at all.
    \end{itemize}
    \item \textbf{Error handling:} A
user must be informed when the input
is malformed or when division by zero
occurs.
    \item \textbf{History:} A user
should be able to view previously
solved equations.
\end{enumerate}
\section{Supported Equations}
The solver is able to handle the
classes of equations shown below.
\begin{itemize}
    \item \textbf{Single unknown on
one side:}
    \begin{equation*}
        2x + 5 = 15 \qquad \Rightarrow
\qquad x = 5
    \end{equation*}
    \item \textbf{Unknown on both
sides:}
    \begin{equation*}
        3x - 4 = x + 6 \qquad
\Rightarrow \qquad x = 5
    \end{equation*}
    \item \textbf{Parenthesised
terms:}
    \begin{equation*}
        2(x + 3) = 10 \qquad
\Rightarrow \qquad x = 2
    \end{equation*}
    \item \textbf{Fractional
coefficients:}
    \begin{equation*}
        \frac{x}{2} + 1 = 4 \qquad
\Rightarrow \qquad x = 6
    \end{equation*}
    \item \textbf{Simultaneous
equations in two unknowns:}
    \begin{equation*}
        x + y = 5, \quad 2x - y = 1
\qquad \Rightarrow \qquad x = 2, \; y
= 3
    \end{equation*}
\end{itemize}
Quadratic, trigonometric and
transcendental equations lie outside
the scope of this draft.
\section{Software Attributes}
Our calculator application has the
following software attributes:
\begin{itemize}
    \item \underline{Functionality}:
arithmetic evaluation, linear equation
solving and history retrieval.
    \item \underline{Reliability}:
every solution is verified by
substitution before it is displayed to
the user.
    \item \underline{Usability}:
equations are entered in ordinary
mathematical notation, requiring no
specialised syntax.
    \item \underline{Portability}: the
application is deployed both as a
desktop program and as an Android
mobile application.
\end{itemize}
\section{Conclusion}
This draft establishes the scope of a
calculator that treats an equation as
a first-class input rather than a
sequence of keystrokes. By restricting
the solver to linear forms, the system
remains small enough to specify
precisely while still covering the
equations most commonly encountered by
its intended users. Subsequent
revisions may extend the grammar to
higher-degree polynomials once the
linear case has been validated.
\end {document}
```

Chapai - A Print Management System

Muhammad Ahmed Qazi

September 2026

Abstract

This article specifies the software requirements for a smart printing management system to be deployed in a university campus environment.

1 Introduction

Traditional paper document printing architectures on university campuses represent a significant logistical bottleneck, operational inefficiency, and economic vulnerability.

To overcome these dificiencies, "Chapai" introduces an automated, transactionally secure, hardware-aware, and priority-driven smart campus printing management network.

1.1 User Requirements

Following are the requirements that our system fulfils:

1. **Authentication:** A user should be able to create an account and login to it.
2. **Print:** A user (student or faculty) must be able to send a document to be printed with the desired configuration to our printing kiosks.
3. **Manage credit:** A user should be able to do the following:
   - See the balance they have in their account.
   - Add some balance to their account.
   - See their transaction history.
4. **Tracking:** A user should be able to track the status of their document whether it has been queued, being printed, or ready for collection.

2 Backend and Front-End Tools

Our system uses the following tools to lay the foundation of our system, i.e. *back-end tools*:

- **Database Management System:** *Oracle Database 21c Express Edition (XE)* with *Oracle SQL Developer as GUI.*
- **Backend Architecture:** *NestJS Framework*, an enterprise-grade TypeScript framework built atop Node.js, implementing a strict modular structure and managing standard Oracle client database drivers via optimised persistence layers.
- **Containerization Subsystem:** Podman Container Engine, utilised to sandbox the Oracle databse environment, ensuring complete reproducibility across developer workstations and preventing operating system dependency pollution.

The following tools are used to build the interfaces for our system, i.e. *front-end tools*:

- **Web Interface:** *Angular Framework.*
- **Mobile Interface:** *Ionic Framework* combined with *Angular 17.*

3 Software Attributes

Our print management system has the following software attributes:

- Functionality: print job submission, authentication, account balance and transactions.
- Usability: the system has two distinct interface for a user and an operator which are both designed to accommodate the technical ability of the user.
- Portability: this system is both deployed as a desktop website and an Android mobile application.

4 Conclusion

The Chapai platform successfully demonstrates a highly functional, enterprise-grade, three-tier application designed to modernize and streamline printing operations across the NED University campus. By replacing inefficient, manual printing queues and unverified cash transactions with a centralized digital ledger and intelligent hardware routing, the system addresses core logistical bottlenecks for both students and administration.

The platform heavily leverages advanced capabilities inherent to the Oracle 21c ecosystem. Complex business operations, such as simultaneous balance deductions and ledger updates, are securely encapsulated inside PL/SQL stored procedures, guaranteeing strict transactional atomicity. Database triggers autonomously maintain referential integrity across unlinked relational bridges, while highly optimized SQL views dynamically flatten complex user hierarchies to provide real-time analytical reporting. By enforcing data isolation at the database layer and controlling access through a robust NestJS API, the architecture ensures a highly secure and highly available production environment.

Basic Functionality of an IoT-Based Health Monitoring System

Muhammad Ahmed Qazi

September 2026

Contents

1 Introduction **2**

2 System Architecture and Functionality **3**
2.1 System Workflow 3
2.1.1 Data Acquisition 4
2.1.2 Data Processing and Transmission 4
2.1.3 Cloud Storage and Analytics 4

3 Advantages and Applications **5**
3.1 Key Advantages 5
3.2 Primary Applications 5

4 Conclusion **6**

Chapter 1

Introduction

An **Internet of Things (IoT) based Health Monitoring System** is an interconnected network of smart devices designed to **continuously track, transmit, and analyze** patient health metrics in real time. This technology serves as a vital bridge in modern healthcare, delivering key benefits across several critical areas:

- **Remote Patient Monitoring:** Enables continuous clinical oversight from the comfort of a patient's home.
- **Chronic Disease Management:** Provides long-term tracking for health conditions requiring persistent vigilance.
- **Elderly Care:** Enhances independence and safety for senior citizens without requiring frequent, stressful hospital visits.

Chapter 2

System Architecture and Functionality

2.1 System Workflow

The basic functionality of the system relies on a seamless flow of data from the patient to the healthcare provider through four main operational stages.

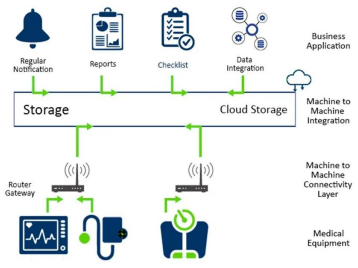


Figure 2.1: This is the image description.

2.1.1 Data Acquisition

Biological sensors—either wearable or implantable—continuously measure vital physiological parameters from the patient:

- **Body Temperature Sensors:** Track sudden thermoregulatory fluctuations or fever spikes.
- **Pulse Oximeters:** Measure heart rate and blood oxygen saturation levels.
- **Electrocardiogram (ECG) Sensors:** Monitor cardiac rhythm and detect potential arrhythmias.

2.1.2 Data Processing and Transmission

Raw physiological signals gathered by the sensors are directed to a central processing unit:

- **Microprocessing Unit:** Hardware such as the *ESP32, Arduino, or Raspberry Pi* filters, calibrates, and digitizes raw biological signals.
- **Communication Protocols:** Processed data is transmitted securely via *Wi-Fi, Bluetooth Low Energy (BLE), or Zigbee* to a local gateway or directly to the internet.

2.1.3 Cloud Storage and Analytics

Once transmitted, the data enters a secure cloud infrastructure:

- **Historical Storage:** Patient health metrics are archived securely to build accessible, long-term medical records.
- **Automated Analytics:** Machine learning algorithms process arriving data streams in real time to detect subtle anomalies, such as irregular heart rhythms or acute stress spikes.

Chapter 3

Advantages and Applications

3.1 Key Advantages

Integrating IoT architecture into medical monitoring provides several transformative advantages:

- **Reduced Hospital Readmissions:** Proactive monitoring prevents medical complications before they escalate into acute emergency stays.
- **Cost Efficiency:** Decreases overall healthcare expenditures by minimizing unnecessary in-person consultations and travel.
- **Proactive Care Paradigm:** Shifts medical practice from reactive treatment to continuous, preventive intervention.

3.2 Primary Applications

The system demonstrates significant utility across diverse medical domain scenarios:

- **Elderly Remote Care:** Supports senior citizens living independently by tracking vitals and providing continuous emergency signaling.
- **Cardiovascular Monitoring:** Tracks heart rate variability and blood pressure for high-risk cardiac patients.
- **Post-Operative Recovery:** Ensures patients recovering from surgical procedures remain stable following early hospital discharge.

Chapter 4

Conclusion

In summary, an IoT health monitoring system automates vital sign monitoring to establish continuous vigilance over patient health. By uniting sensor technology, wireless networking, and cloud computing, it provides an efficient, cost-effective, and deeply proactive foundation for modern digital healthcare.

Requirement Specification for a Linear Equation Calculator

Muhammad Ahmed Qazi

September 2026

Abstract

This article presents an initial draft of the software requirements for a calculator application capable of performing arithmetic operations and solving basic linear equations.

1 Introduction

Conventional pocket calculators evaluate arithmetic expressions but cannot resolve an unknown quantity presented in equation form. The proposed calculator extends standard arithmetic functionality with a symbolic solver for basic linear equations, allowing a user to enter an equation directly and obtain its solution.

1.1 User Requirements

Following are the requirements that our system fulfils:

1. **Arithmetic:** A user must be able to evaluate expressions involving addition, subtraction, multiplication and division.
2. **Equation entry:** A user must be able to enter an equation containing an equality sign and one or more unknowns.
3. **Solving:** The system must determine the value of each unknown and report the following:
   - A unique solution, where one exists.
   - That the equation holds for every value of the unknown.

   - That the equation has no solution at all.
4. **Error handling:** A user must be informed when the input is malformed or when division by zero occurs.
5. **History:** A user should be able to view previously solved equations.

2 Supported Equations

The solver is able to handle the classes of equations shown below.

- **Single unknown on one side:**

$$2x + 5 = 15 \qquad \Rightarrow \qquad x = 5$$

- **Unknown on both sides:**

$$3x - 4 = x + 6 \qquad \Rightarrow \qquad x = 5$$

- **Parenthesised terms:**

$$2(x + 3) = 10 \qquad \Rightarrow \qquad x = 2$$

- **Fractional coefficients:**

$$\frac{x}{2} + 1 = 4 \qquad \Rightarrow \qquad x = 6$$

- **Simultaneous equations in two unknowns:**

$$x + y = 5, \quad 2x - y = 1 \qquad \Rightarrow \qquad x = 2,\ y = 3$$

Quadratic, trigonometric and transcendental equations lie outside the scope of this draft.

3 Software Attributes

Our calculator application has the following software attributes:

- Functionality: arithmetic evaluation, linear equation solving and history retrieval.

- Reliability: every solution is verified by substitution before it is displayed to the user.
- Usability: equations are entered in ordinary mathematical notation, requiring no specialised syntax.
- Portability: the application is deployed both as a desktop program and as an Android mobile application.

4 Conclusion

This draft establishes the scope of a calculator that treats an equation as a first-class input rather than a sequence of keystrokes. By restricting the solver to linear forms, the system remains small enough to specify precisely while still covering the equations most commonly encountered by its intended users. Subsequent revisions may extend the grammar to higher-degree polynomials once the linear case has been validated.